

avito.tech



64K+

Охват за 30 дней

AvitoTech

У нас живут ваши объявления

4,39 Оценка работодателя

557,93 Рейтинг

194 450 Подписчики

[Подписаться](#)**mariklolik**

19 часов назад

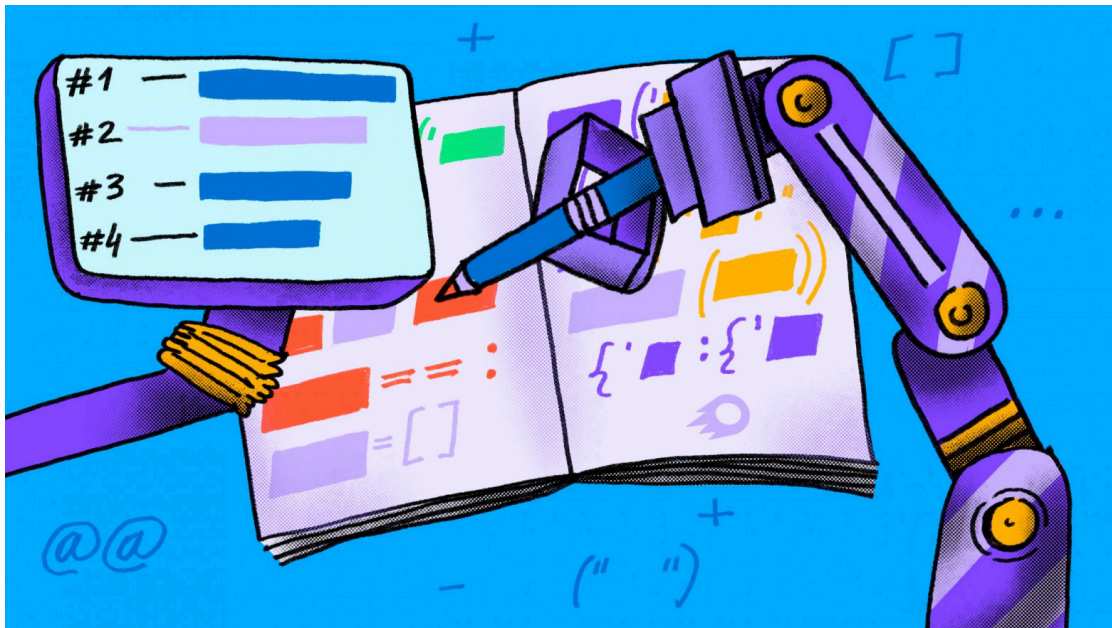
Как переложить нагрузку по code review с разработчиков на LLM[Средний](#) [7 мин](#) [7.5K](#)

Блог компании AvitoTech, Natural Language Processing*, Big Data*, Искусственный интеллект, Проектирование и рефакторинг

[Кейс](#)

Привет! Меня зовут **Марк Каширский**, я работаю DS-инженером в команде LLM [Авито](#). Создаю инструменты для разработчиков, чтобы им было легче и удобнее работать. В статье рассказываю, как мы автоматизировали процесс Code review при помощи больших языковых моделей.

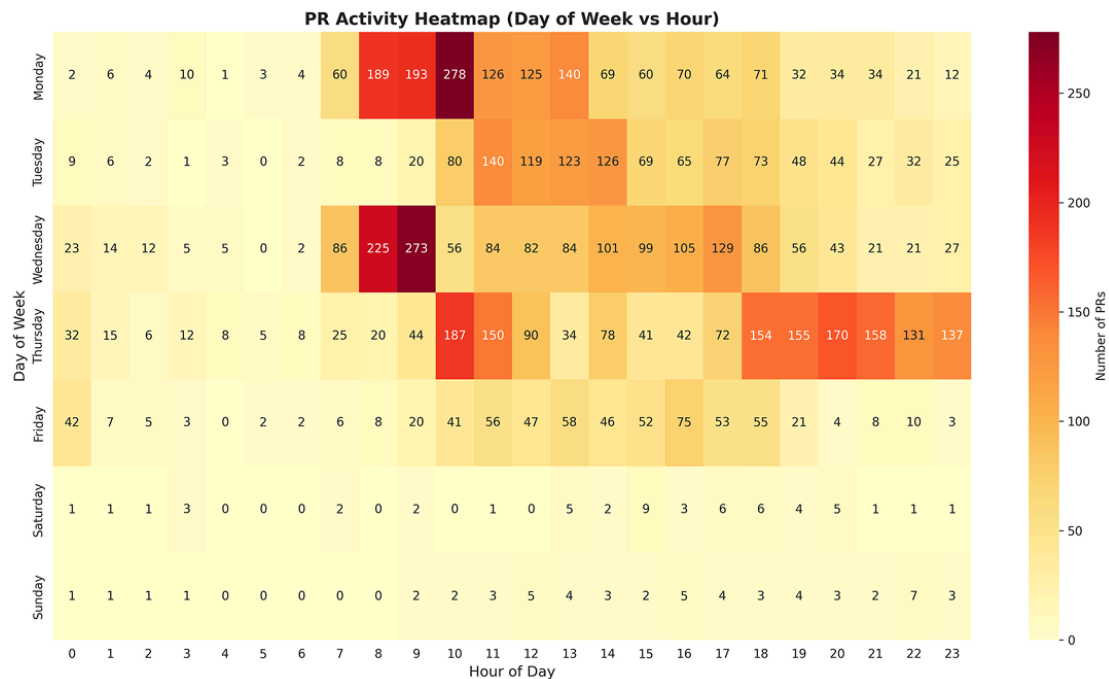
В этой статье[Причины для автоматизации процесса Code review](#)[Архитектура системы Code review](#)[Выбор LLM-модели](#)[Этапы ML-пайплайна](#)[Метрики решения](#)[Вместо выводов](#)



Причины для автоматизации процесса Code review

Ревью кода — рутинный процесс с одинаковым набором действий, который может тормозить выпуск фичей. Мы предположили, что можно сильно ускорить доставку кода до продакшена и уменьшить нагрузку на разработчиков, если переложить задачу ревью на ИИ.

Мы провели большое исследование на исторических данных, чтобы выяснить, сколько **пул-реквестов (PR)** создаётся в Авито. Получилось 1000 пул-реквестов в неделю, в пиковые часы — до 300 штук в час.



На карте видно, что в середине недели наибольшая нагрузка на ревьюеров

Основная задача — сократить нагрузку на разработчиков и ревьюеров. Но это была не единственная мотивация, всего мы сформулировали для себя три основных цели.

Сократить время на разработку новых фичей. Разработчикам приходится постоянно отвлекаться от основной работы, чтобы переключаться на Code review. При тысячах PR в неделю

это создаёт большую нагрузку на команду. Пул-реквесты могут долго находиться в статусе «Ревью» просто потому, что их не успевают посмотреть. В результате увеличивается time-to-market новых фичей. Мы решили ускорить этот процесс.

Снизить объём влияния человека. При постоянном переключении на Code review разработчики устают, из-за чего могут пропустить неочевидные баги и участки кода, критичные для безопасности.

Унифицировать процесс ревью и поддерживать высокое качество кода. Подход к комментариям у разных ревьюеров отличается, часто возникают дискуссионные моменты и вкусовщина. Мы хотим сделать Code review унифицированным инструментом и поддерживать единое качество кода, а ответственность за это переложить на модель.

[Тут еще больше контента](#)

Архитектура системы Code review

Верхнеуровнево система состоит из трёх компонентов:

- Stash — интерфейс, через который разработчики запускают ревью
- Go-сервис — регистратор событий и оркестратор процесса
- Python ML Pipeline service — сервис для генерации комментариев

Разработчику достаточно в любом PR оставить комментарий ai_review. После этого сгенерируется событие, которое отправится в Data Bus, и уже оттуда его вычитает Go-сервис

Go-сервис взаимодействует со Stash, вычитывает события и управляет состояниями ревью. Так как ревью для больших PR может занимать много времени, мы отдельно выделили ML-сервис, внутри которого и живет вся магия LLM. ML-сервис взаимодействует с Go-сервисом, передает обновленные состояния событий и сгенерированные задачи. После этого Go-сервис публикует их на Stash.

End-to-End поток запроса. В конце генерируются итоговые комментарии, которые уходят в Go-сервис и публикуются на Stash

Процесс по шагам

1. Запуск ревью из Stash. Пользователь оставляет комментарий ai_review, после чего генерируется Data Bus событие. Go-сервис получает его и парсит команду.

2. Подготовка данных. Go-сервис получает diff и такие метаданные, как структура репозитория и существующие комментарии. Далее кладёт ревью в базу со статусом STATE_REQUESTED. После этого отправляет RPC в ML-сервис.

3. Вызов ML-сервиса (RPC). Поскольку генерация может длиться долго, ML-сервис немедленно отвечает 200 OK, чтобы не блокировать Go-сервис и долго не держать соединение. После этого ревью переходит в статус BackgroundTaskRunner.

Выбор LLM-модели

Перед нами стояла новая задача, для которой существует не так много бенчмарков. Одним из самых релевантных для нас оказался MERA-RuCodeReviewer. Бенчмарк представляет собой 689 примеров изменений кода на языках Java, Python, Scala и Go. Все эти языки активно используются в нашей разработке.

В лидерборде моделей первое место с большим отрывом занимает Gemini 2.5 flash, а второе — Qwen3-Coder-30B-Instruct-FP8. Мы остановились на второй модели: она опенсорсная и позволяет хостить решение на собственной инфраструктуре, что для нас очень важно. Дополнительно составили таксономию самых частотных ошибок, которые возникали в исторических PR, чтобы дообучить модель.

Лидерборд моделей MERA-RuCodeReviewer: мы выбрали Qwen3-Coder-30B-Instruct-FP8 и она занимает второе место

Две причины селф-хостинга

Безопасность. Важно, чтобы весь код оставался во внутреннем контуре и не передавался во внешние системы через API.

Экономика. По нагрузке генерации получают токенов: для небольших PR это около 2 000 токенов, а для крупных — до 10 000. При таких объёмах использование API становится дорогим, тогда как собственные ресурсы у нас уже есть. По сравнению с API-моделью селф-хостинг позволяет снизить стоимость без потери качества.

[Жми сюда!](#)

Этапы ML-пайплайна

ML-пайплайн включает три основных этапа. Разберем каждый подробнее, но сначала обсудим подготовку.

Подготовительный этап. Перед началом процессинга необходимо аннотировать diff. Стандартный git diff не содержит абсолютных номеров строк и статуса, поэтому требуется более расширенный контекст. ML-модели нужен не только diff, но и полный файл в актуальном состоянии. Go-сервис запрашивает его из Stash через RPC и передает в Python-сервис для дальнейшей обработки.

DO (Raw Diff). Git diff не содержит абсолютных номеров строк, поэтому точно комментировать невозможно.

```
...
@@ -45,7 +45,9 @@ def process_data(data):
if data is None:
- return None
+ raise ValueError ("Data cannot be None")
...
```

[Объяснить с](#)  SourceCraft

ПОСЛЕ (Annotated). Скрипт 'annotate_diff_bitsai cr' добавляет номера строк для нового и старого кода.

```
...
@@ -45,7 +45,9 @@ def process_data(data):
[unchanged] if data is None:
[deleted or pre-modified @46 in old code] - return
None
[added or post-modified @46 in new code] + raise
ValueError ("Data cannot be None")
...
```

[Объяснить с](#) 

Этап 1. RuleChecker. Генерация. Это модель, задача которой — выявлять все потенциальные проблемы в коде. Она оптимизирована на высокую полноту при низкой точности, чтобы обеспечить максимальное покрытие возможных проблем. В основе модели лежит заранее сформированная на основе наших репозиториях таксономия проблем. Мы проанализировали, какие проблемы чаще всего встречаются в реальных задачах, и обучили модель распознавать их. Выделили четыре класса:

- безопасность;
- дефекты разработки;
- перфоманс;
- поддержка кода.

Итог этапа — модель генерирует JSON-массив, где для каждого замечания сохраняются:

- строка кода, к которой относится комментарий;
- категория проблемы;
- текст замечания;
- рекомендация по исправлению.

Этап 2. ReviewFilter. Валидация. На этом этапе мы повышаем точность, чтобы не перегружать разработчиков избыточными комментариями. Фильтр отсеивает false positives и нерелевантные замечания, оставляет только важные проблемы. При этом модель генерирует краткое обоснование, почему она посчитала комментарий важным или нет.

Модель работает в паттерне Conclusion First. Мы передаем ей тот же контекст (diff и полный файл), а также конкретный комментарий, привязанный к строке кода. Задача модели — ответить «да» или

«нет» на вопрос «является ли это замечание действительно важным». Все комментарии, которые получают «нет», мы отбрасываем.

Этап 3. CommentAggregator. Дедупликация. Мы строим эмбединги комментариев с помощью модели BAAI/bge-m3 и вычисляем их семантическую близость. Если cosine similarity комментариев превышает 0.85, они группируются. Внутри каждой группы (при условии, что комментарии относятся к одной строке) мы оставляем один наиболее содержательный и корректный. В результате формируется финальный JSON со списком уникальных комментариев, который передается в Go-сервис и далее отображается пользователю.

Финал. Python-сервис завершает процесс ревью и вызывает RPC у Go-сервиса. Go-сервис устанавливает статус STATE_FINISHED и после этого через API публикует все комментарии, сгенерированные моделью. Именно их в итоге видит разработчик в интерфейсе — без дублирующихся комментариев и лишней информации.

Детальный ML-пайплайн, на котором описаны все этапы

Производительность решения. На демо-репозиториях мы собираем усредненный diff. Пул-реквесты условно делим на малые — до 3 файлов и большие — 10+ файлов. Малые пул-реквесты обрабатываются за 10–20 секунд. Большие — примерно за 60 секунд.

Для стабильной работы мы ограничиваем количество параллельных ревью до 650 штук. Это число получили из анализа нагрузки, и специально дополнительно зависили, чтобы оставить ресурс на расширение.

Метрики решения

Мы собираем несколько метрик, чтобы оценивать качество комментариев, генерируемых моделью, и влияние на процесс ревью.

Precision на уровне 85%. Каждому комментарию пользователь может поставить лайк или дизлайк. Лайк означает, что комментарий полезен, дизлайк — что нет.

Пример генерации. В одном из пул-реквестов неправильно указали диапазон времени — не по московскому времени. Модель предложила исправить диапазон, и разработчик поставил лайк

На текущий момент precision агрегировано на уровне 85%. Значит, 85% комментариев разработчики классифицируют как хорошие.

Высокая оценка DS-специалиста

Outdated rate на уровне 32% для всех репозиторий. Эта метрика показывает, насколько комментарии реально помогают исправлять код. Если комментарий был закрыт исправлением кода, мы считаем его полезным. Метрика показывает, что треть наших комментариев действительно помогает разработчикам

Outdated rate можно разделить по категориям репозиторий. Например, в Infra и DevOps метрика достигает 60%, что даже для человеческих комментариев считается высоким результатом

Вместо выводов

→ Автоматизация Code review с использованием больших языковых моделей помогает ускорить процесс разработки и поддерживать высокое качество кода.

→ Модель Qwen3-Coder-30B-Instruct-FP8 показала хорошие результаты по метрикам.

→ Не стоит решать всё одним запросом — лучше разделять генерацию (Recall) и валидацию (Precision). Даже с длинным контекстом модель теряется, поэтому нужна сложная система с разделением этапов.

→ Валидация улучшает качество комментариев. Второй этап модели ReviewFilter отсекает 90% галлюцинаций. Это критично для доверия пользователей.

→ Модели необходимо дать доступ к полному коду файлов и структуре репозитория — одних изменений в diff недостаточно.

Отметим, что скидывать процесс на AI и оставлять его без надзора нельзя. Передать рутину роботу хорошо и правильно, но контролировать процесс всё равно должны люди.

Больше про DS и ML рассказываем в нашем телеграм-канале [«Доска AI-объявлений»](#). Пишем про ИИ, делимся вакансиями и анонсируем интересные мероприятия.

→ [Заглянуть](#)

[Клики здесь и узнаешь](#)

Теги: [review](#), [pull request](#), [llm](#), [agents](#), [llm-агент](#)

Хабы: [Блог компании AvitoTech](#), [Natural Language Processing](#), [Big Data](#), [Искусственный интеллект](#), [Проектирование и рефакторинг](#)



Редакторский дайджест

Присылаем лучшие статьи раз в месяц

[Подписаться](#)

Оставляя почту, я принимаю [Политику конфиденциальности](#) и даю согласие на получение рассылок



64K+

Охват за 30 дней

AvitoTech

У нас живут ваши объявления



4K+

1

Охват за 30 дней Карма

Марк Каширский @mariklolik

Пользователь

Подписаться



Комментарии 6

Публикации

ЛУЧШИЕ ЗА СУТКИ

ПОХОЖИЕ



the_bat

23 часа назад

Ремонт блока питания с Power Delivery. 470 граммов электроники

Простой

4 мин

16K



+97



31



27



interpres

18 часов назад

Электроинструмент становится хуже, и это делается намеренно

Простой

11 мин

27K

Обзор

Перевод



+72



45



85



prohronus

19 часов назад

Как ИИ-агенты стали новым оружием скамеров на Хабр Карьере

5 мин

9.3K

Кейс



+52



9



9



alizar

22 часа назад

Готовимся к отключению. Эффективные форматы для упаковки и раздачи HTML-страниц

Простой

7 мин

12K

Обзор



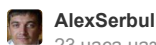
+37



59



18



AlexSerbul

23 часа назад

Программирование с AI-ассистентом — похороните меня под плитусом

🔗 Средний

🕒 14 мин

👁 10K

Мнение

📦 +28

🔖 33

💬 11



TrexSelectel

19 часов назад

Linux 7.0 и что изменилось в ядре после очередного цикла разработки

🕒 5 мин

👁 8.8K

Обзор

📦 +27

🔖 10

💬 0



mariklolik

19 часов назад

Как переложить нагрузку по code review с разработчиков на LLM

🔗 Средний

🕒 7 мин

👁 7.5K

Кейс

📦 +23

🔖 23

💬 6



nick_dyuba

23 часа назад

Легенды 90-х — кто придумал и производил жвачки Turbo, Love is... и TіріTіr

🕒 5 мин

👁 7.1K

Recovery Mode

📦 +22

🔖 8

💬 5



Mimizavr

23 часа назад

«Великое очищение» в работе с контентом: что осталось от роли редактора

🕒 11 мин

👁 7K

📦 +17

🔖 9

💬 7



infgotoinf

11 часов назад

Perl — зря забытый язык программирования?

🕒 11 мин

👁 8.6K

Из песочницы

📦 +16

🔖 13

💬 10

Показать еще

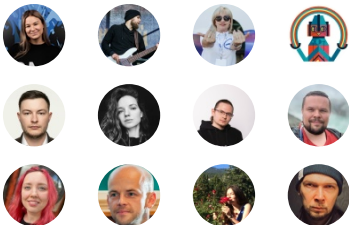
ИНФОРМАЦИЯ

Сайт	avito.tech
Дата регистрации	27 мая 2016
Дата основания	2007
Численность	5 001–10 000 человек
Местоположение	Россия
Представитель	vvroschin

ВКОНТАКТЕ



11,671 followers

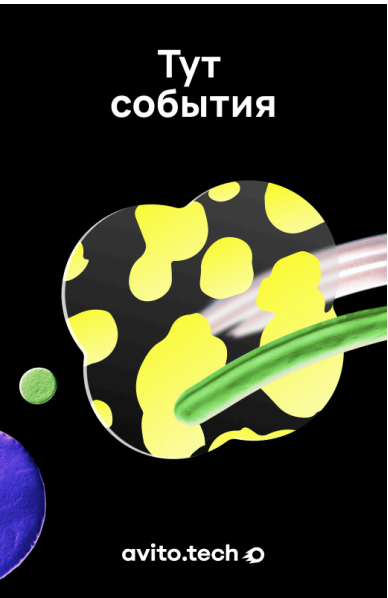


Follow on VK

ВИДЖЕТ



ВИДЖЕТ



ВИДЖЕТ



БЛОГ НА ХАБРЕ

19 часов назад

Как переложить нагрузку по code review с разработчиков на LLM

7.5K 6

16 апр в 22:20

Фитнес план для PostgreSQL своими руками: как похудеть, когда нет доступа к pg_repack и pgcompacttable

11K 15

14 апр в 17:15

Как я искал самую эргономичную клавиатуру

16K 98

10 апр в 18:16

Как работает система фейков для сквозного тестирования в Авито

7.1K 0

8 апр в 17:15

Одна за всех? Как я организовала более 100+ встреч QA-комьюнити и не выгорела

6.7K 0

Ваш аккаунт

Войти

Регистрация

Разделы

Статьи

Новости

Хабы

Информация

Устройство сайта

Для авторов

Для компаний

Услуги

Корпоративный блог

Медийная реклама

Нативные проекты

[Компании](#)

[Авторы](#)

[Песочница](#)

[Документы](#)

[Соглашение](#)

[Конфиденциальность](#)

[Образовательные программы](#)

[Стартапам](#)



[Настройка языка](#)

[Техническая поддержка](#)

© 2006–2026, Habr